LLM Agents - Tools

Autonomous Software Agents

A.A. 2025-2026

Prof. Paolo Giorgini



UNIVERSITY OF TRENTO - Italy
Department of Information
and Communication Technology

Why Agents Require Stronger Models

Compound mistakes

- Agents solve tasks through multiple sequential steps
- Errors accumulate across steps
- Eg. 95% accuracy per step:
  - 10 steps → ~60% overall accuracy
  - 100 steps → ~0.6% overall accuracy

Higher stakes

- Agents can access tools and perform impactful actions
- Failures may have real and severe consequences
  - Wrong database updates
  - Incorrect automated decisions
  - Costly system actions

Cost, Autonomy, and Trade-offs

Cost of multi-step tasks

- Take more time
- Consume more compute
- Cost more money

Why agents are still worth it

- Save large amounts of human time
- Automate complex workflows
- In many scenarios, the benefits outweigh the costs

Key trade-off

- Agents are expensive to run, but even more expensive to replace with humans

What Determines Agent Success

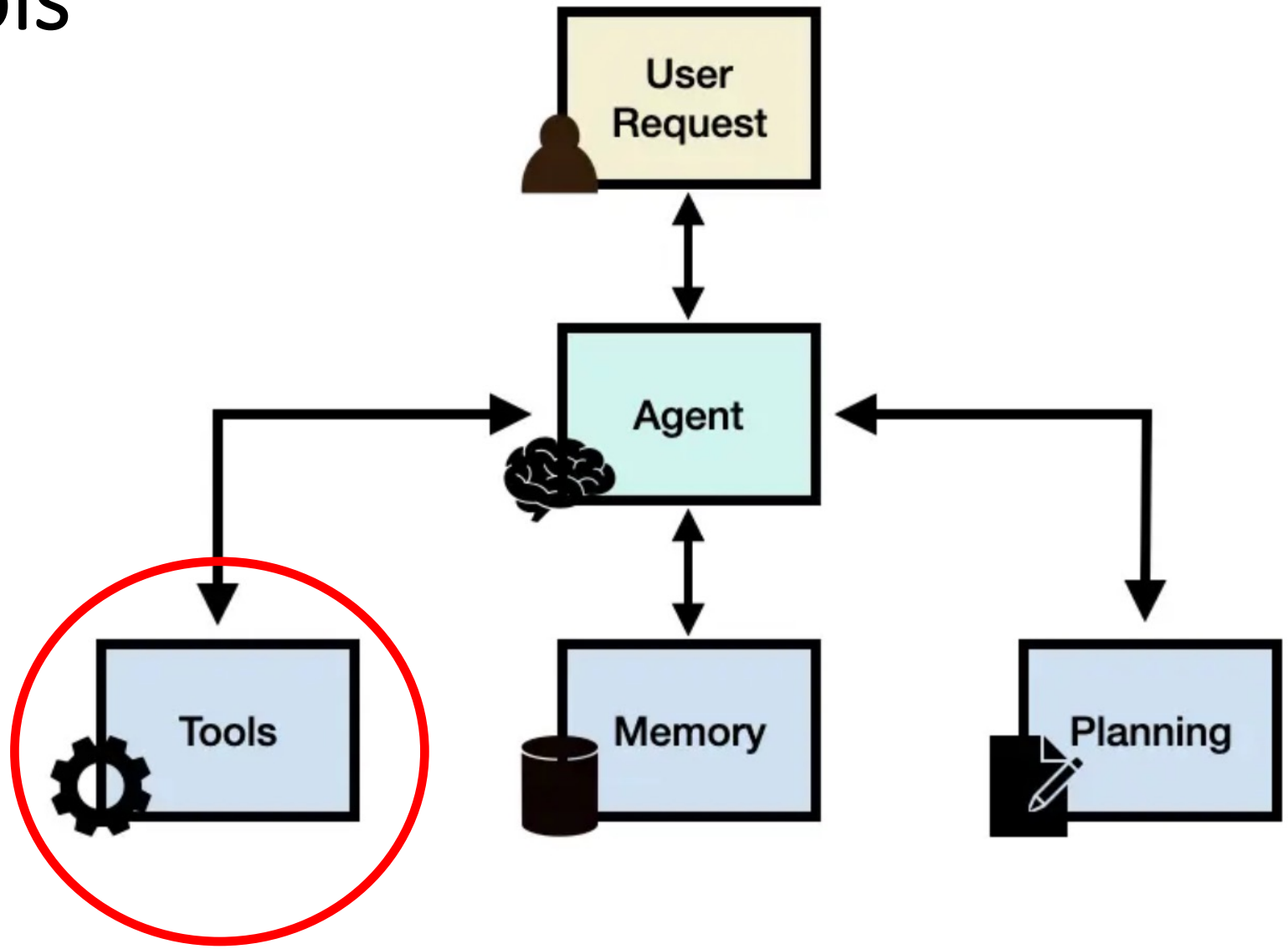
Given a fixed environment, an agent's success depends on:

- Tool inventory
  - What tools are available?
  - How expressive and reliable are they?
- Planner strength
  - Plan actions
  - Decide what to do next
  - Recover from partial failures

Strong agents are defined as much by their tools and planning as by the model itself

- To understand agent performance, we must first understand the tools they can use`

Tools

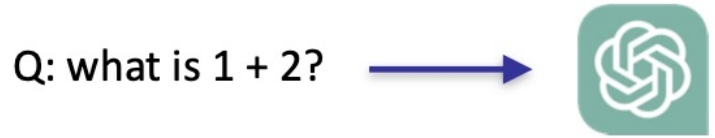


LLM agents' Tools

- An agent is defined by **where it operates** and **what it can do**
- Agents are not defined only by intelligence, but by their environment and the **tools** they can use
- Many modern AI systems are **tool-using agents**
  - Eg, ChatGPT: web search, code execution, image generation
  - Eg, RAG systems: text retrievers, image retrievers, SQL executors
- The environment determines which tools are potentially usable
- The available tools determine which environments the agent can effectively operate in

Question answering

Q: what is 1 + 2?



A: 3

Q: Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for $2 per egg. How much does she make every day?



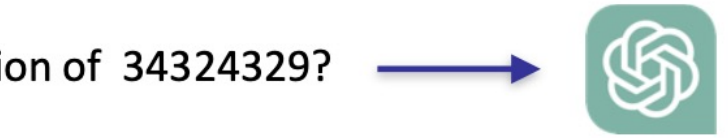
Requires reasoning

Q: who is the latest UK PM?



Requires knowledge

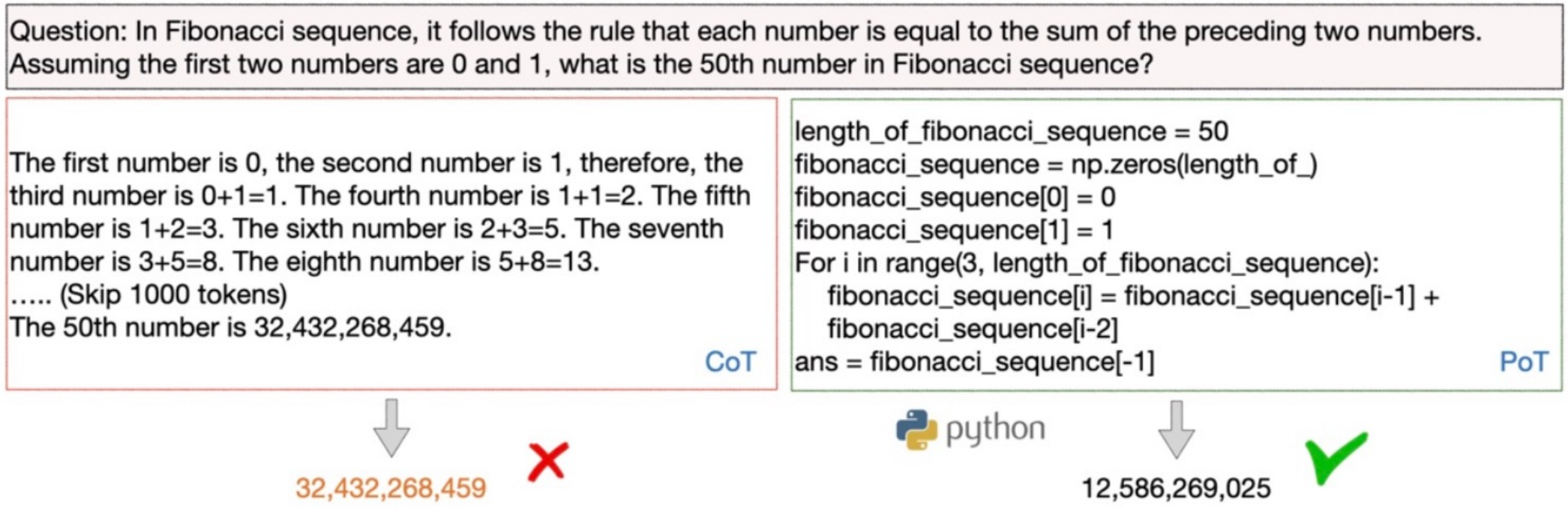
Q: what is the prime factorization of 34324329?



Requires computation

Shunyu Yao

Code augmentation for computation



Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks

Tool use

- Special tokens to invoke tool calls for
  - Search engine, calculator, etc.
  - Task-specific models (translation)
  - APIs
- Unnatural format requires task/tool-specific fine-tuning
- Multiple tool calls?

A weather task:
how hot will it get in NYC today? |*weather* lookup region=NYC |*result* precipitation chance: 10, high temp: 20c, low-temp: 12c |*output* today's high will be 20C

TALM: Tool Augmented Language Models.

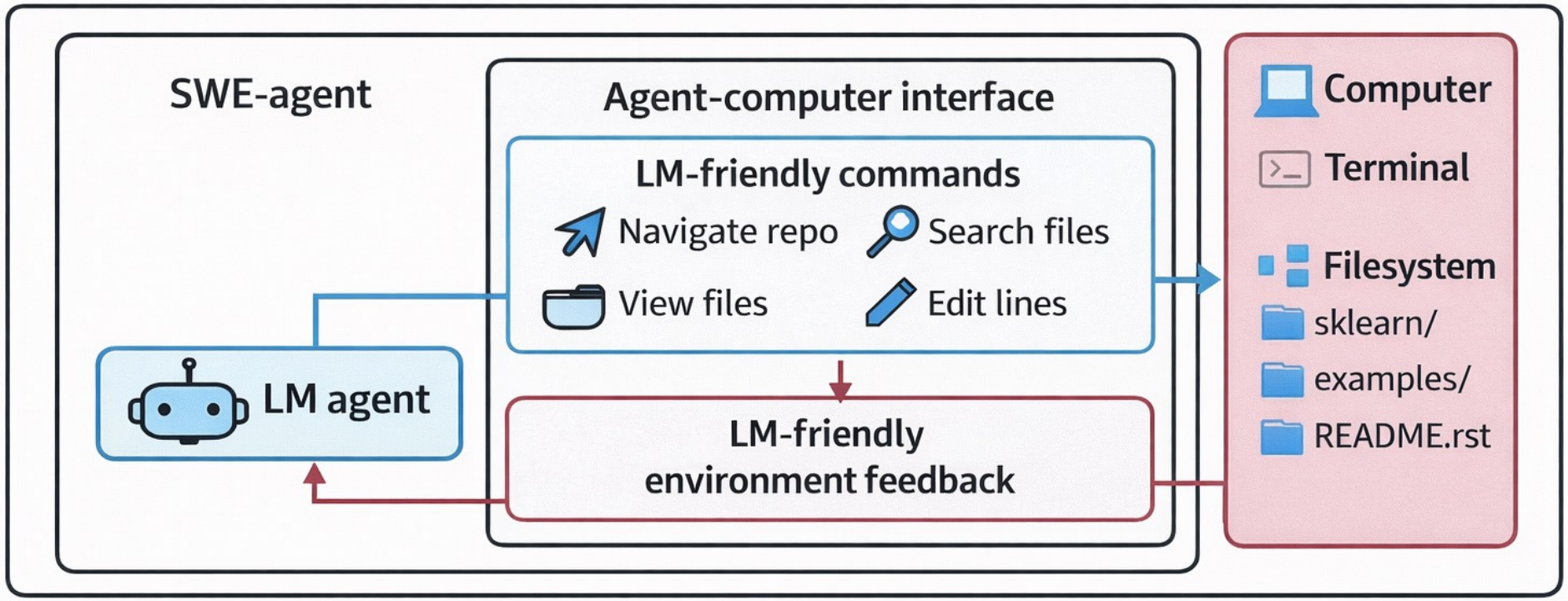
Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Toolformer: Language Models Can Teach Themselves to Use Tools

- **SWE-agent** (Yang et al, 2024) is an agent built on top of GPT-4 that operates in a computer environment. Its environment includes the terminal and the file system, and its actions include navigating repositories, searching files, viewing files, and editing code



Tools and the Role of AI Agents

- An AI system **does not need external tools** in order to be considered an agent
  - However, **without tools**, an agent's capabilities are **severely limited**
- A standalone model usually performs **one type of action**:
  - LLM → text generation
  - Image generator → image generation
- **External tools transform a model into a capable agent**
  - They expand what the agent can perceive and what it can do
  - Tools are therefore a **key enabler of agentic behavior**

Tools for Perception and Action

- Tools allow an agent to both:
  - **Perceive the environment**
  - **Act upon the environment**
- Two fundamental categories of actions:
  - **Read-only actions**
    - Retrieve information
    - Observe the current state of the environment
  - **Write actions**
    - Modify data
    - Trigger real-world effects
- Examples:
  - Reading from a database → read-only
  - Updating a database or sending an email → write

Tools act as the **interface between the model and the environment**

Tool Inventory and Tool Selection

- The set of tools an agent can access is called its **tool inventory**
  - The tool inventory determines **what the agent is capable of doing**
- Trade-off:
  - More tools ⇒ more capabilities
  - More tools ⇒ harder to understand and use them effectively
- Designing a good tool inventory requires:
  - **Careful reasoning**
  - **Experimentation**
- Choosing the right tools is known as **tool selection**

Knowledge Augmentation Tools

- Knowledge augmentation tools provide the agent with **relevant context**
- Purpose:
  - Improve response quality
  - Reduce hallucinations
  - Examples :
    - Text retrievers, Image retrievers, SQL executors, Email readers, Internal people search, Inventory APIs
- These tools can access:
  - **Private organizational data**
  - **Public information from the internet**
- Web browsing is especially important to prevent **model staleness**
  - Weather, news, events, stock prices, flight status, etc.

Capability Extension Tools

- Capability extension tools address **inherent weaknesses of AI models**
- Simple but powerful examples:
  - Calculator (models are bad at arithmetic), Unit converters (e.g., lbs to kg), Timezone converters, Language translators
- Advanced tools:
  - **Code interpreters**
    - Execute code
    - Analyze failures
    - Run experiments
  - Generate charts and graphs
- Tools can turn:
  - Text-only models → multimodal agents
  - By adding image generation, OCR, transcription, LaTeX rendering

Function Calling: Turning Models into Agents

- Seeing a **tool as a function**
  - Invoking a tool can referred as **function calling**

Although APIs differ, the general workflow is:

Step 1 — Create a Tool Inventory

- Declare all tools the model may use
- Each tool is described by:
  - Function name (execution entry point)
  - Parameters
  - Documentation (what the tool does and how to use it)

Controlling Tool Usage

Step 2 — Specify Which Tools the Agent Can Use

- Different queries may require different tools
- APIs allow controlling tool usage per query using settings such as:
  - **required**
  - → the model must use at least one tool
  - **none**
  - → the model must not use any tool
  - **auto**
  - → the model decides which tool(s) to use
- Function calling logic is typically expressed in **pseudocode**
  - The same conceptual flow applies across multiple APIs
  - Implementation details depend on the specific provider

Function Calling Example

- **Tool Definitions**
  - Two simple tools are defined:
    - `lbs_to_kg(lbs)` → converts pounds to kilograms
    - `ft_to_meters(ft)` → converts feet to meters
  - Each tool specifies:
    - Name
    - Description
    - Parameter schema (type, required fields)

```
User query: "How many kilograms are 40 pounds?"
```

Model Decision

- Selects the `lbs_to_kg` tool
- Generates the parameter value: `lbs = 40`

```
function: lbs_to_kg
arguments: { "lbs": 40 }
```

- The API guarantees only **valid tools** are called
- But it **cannot guarantee correct parameters**
- Parameter hallucination remains a key challenge

Function calling enables structured interaction with tools, but correctness depends not only on tool selection, but also on accurate parameter generation

```python
def lbs_to_kg(lbs):
    return lbs * 0.45359237

def ft_to_meters(ft):
    return ft * 0.3048
```

Tool definition

```python
lbs_to_kg_tool = FunctionDeclaration(
    name="lbs_to_kg",
    description=""Convert from pounds to kilograms",
    parameters={
        "type":"object",
        "properties":{
            "lbs": {"type":"int", "description": "The value to be converted"}
        },
        "required": ["lbs"]
    }
ft_to_m_tool = FunctionDeclaration(name="ft_to_meters",..)
```

(1) Tool description

```python
messages = [{"role": "user","content:'[USER_QUERY]]]
```

User query

```python
response = model_client.chat.completions.create(
    model=[MODEL_NAME],
    messages=messages,
    tools=[lbs_to_kg_tool, ft_to_m_tool],
    tool_choice="auto",
)
```

(2) This query can use two tools

```
response = ModelResponse(
   finish_reason='tool_calls',
   message=chat.Message(
      content=None,
      role='assistant',
      tool_calls=[
          ToolCall(
              function=Function(
                  arguments={""lbs": 40},
                  name='lbs_to_kg',
                  type='function')
                  )
          ])
      ))
   ))
)
```

The core problem of Tools usage

- Traditionally, tools have been integrated via ad-hoc function calling, prompt-based API descriptions and custom glue code for each tool
  - Tight coupling between model and tools
  - poor reusability
  - difficult auditing and debugging
  - weak safety guarantees
  - no interoperability across frameworks

Model Context Protocol



- **Model Context Protocol (MCP – Anthropic 2024)** is an **open-source, vendor-neutral standard** that defines how AI models (especially LLMs and AI agents) connect to **external tools, data sources, prompts, and workflows** in a standardized, structured way
  - MCP solves the **integration problem** where AI models previously needed **custom connectors for every tool or data source**
- Provides a **standardized protocol** so AI can reliably:
  - Discover available tools and capabilities
  - Access up-to-date, context-rich information
  - Invoke actions securely via a unified interface
- Makes AI agents much more capable by enabling them to interact with **real-world systems** (databases, APIs, apps) in a modular and interoperable way
- Often described as the **“USB-C for AI”** — a universal connection standard that replaces bespoke, ad-hoc integrations with a common, extensible protocol

MCP Architecture (Conceptual View)

Clean separation of responsibilities



- **Agent / LLM** performs reasoning, planning and decision making
  - does not directly call APIs
  - does not know implementation details
- **MCP Client** runs next to the LLM / agent and sends requests / receives responses
  - translates abstract decisions into MCP messages handling tool calls, errors, and structured responses
- **MCP Server** exposes concrete capabilities (tools, resources and prompts) and encapsulates APIs, Databases, file systems, external services
  - acts as a **security and policy boundary**

Tool discovery is fundamental

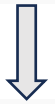
- With MCP:
  - agents **do not assume** which tools exist
  - tools are **discovered dynamically at runtime**
- Why this matters:
  - tool inventories are not hardcoded
  - capabilities can change per task or environment
  - permissions can be enforced centrally
- This directly addresses many **tool selection problems** you've already seen

Tool invocation semantics

1. The agent decides **conceptually** to use a tool
2. The MCP client sends: tool name and structured arguments
3. The MCP server: validates inputs, executes the tool and returns structured results
4. The agent reasons over the result

- Important:
  - errors are **structured objects**
  - not free-form text
  - no fragile parsing

```
Error: file not found, please check path
```

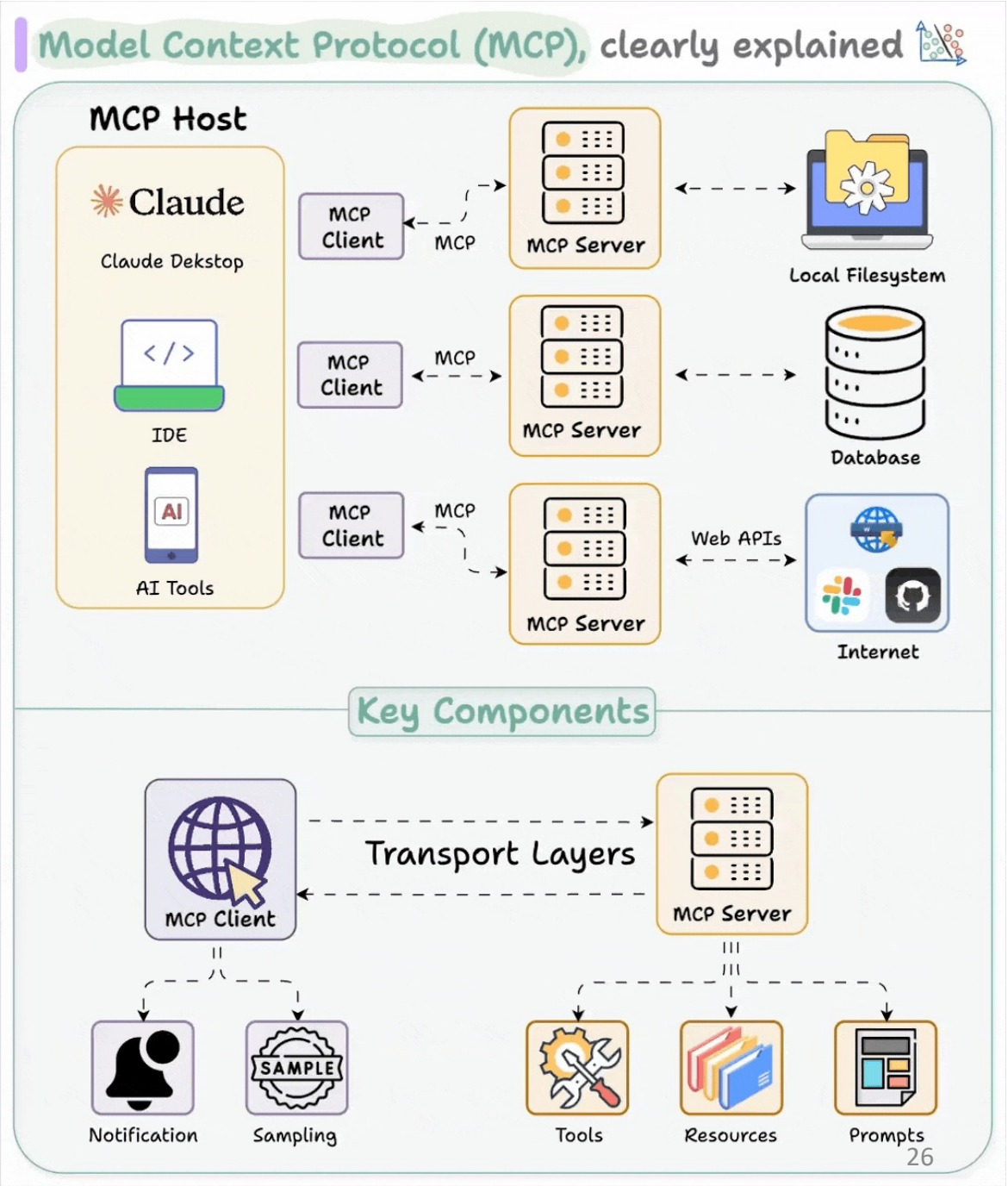


```
{
  "error_type": "FileNotFound",
  "message": "File not found",
  "path": "/data/file.txt"
}
```

The **MCP Host** is the runtime environment in which the agent lives and operates

- Claude Desktop
- VS Code with agent
- A Node.js or Python backend running an agent
- A chat application with agent logic

- An **MCP server** can expose three kinds of entities
  - **Tools** callable functions
  - **Resources** : persistent data sources
  - **Prompts**: usable prompt templates, structured instructions and shared across agents



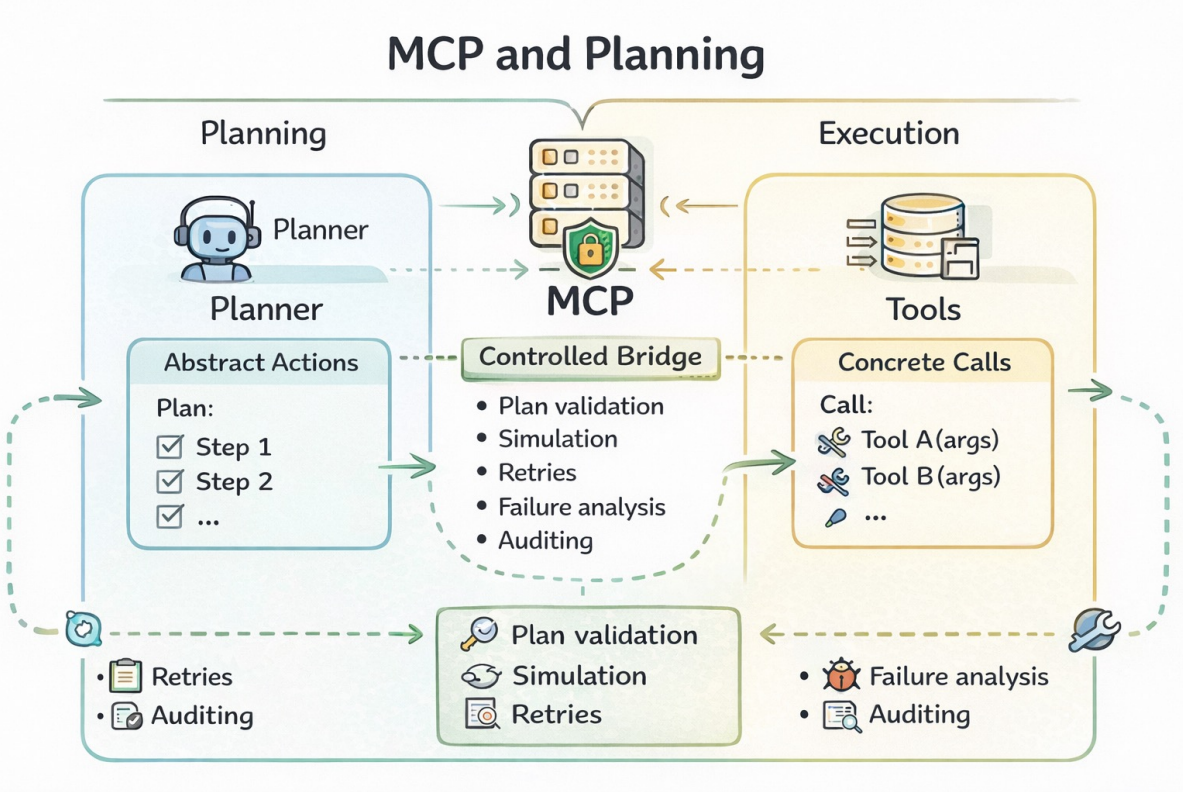
MCP and planning

- Planners generate **abstract action sequences**
- MCP provides the **execution substrate**

Therefore:

  - planning ≠ execution
  - MCP is the **controlled bridge** between the two

- This enables:
  - plan validation
  - simulation
  - Retries
  - Failure analysis
  - auditing



Scenario and Architecture

- **User request** “Send an email to luca@unitn.it with subject *Meeting* and body *Can we meet tomorrow at 10:00?*”

- **Architecture**
  - **MCP Host**: Claude Desktop
  - **Agent**: Claude (LLM + planning logic)
  - **MCP Client**: embedded in Claude Desktop
  - **MCP Server**: Gmail MCP Server
  - **External Service**: Google Gmail API

Agent Planning (inside Claude Desktop)

Claude interprets the request and produces a **plan** based on available tools

```
1. Check available tools via MCP
2. Identify gmail.send_email as required tool
3. Recognize this as a WRITE action
4. Ask user for confirmation
5. If confirmed, execute tool via MCP
6. Report result to user
```

Important

- Planning happens inside the LLM
- No API details are embedded in the prompt
- Tool knowledge is discovered dynamically

MCP Tool Discovery (Claude → MCP Server)

Claude Desktop (MCP Client) queries the MCP Server:

MCP message

```
{
  "jsonrpc": "2.0",
  "method": "tools/list",
  "id": 1
}
```

- Claude learns **what tools exist**
- The **write action** is explicit
- Tool schemas are machine-readable

MCP server response

```
{
  "tools": [
    {
      "name": "gmail.send_email",
      "description": "Send an email via Gmail",
      "inputSchema": {
        "to": "string",
        "subject": "string",
        "body": "string"
      },
      "sideEffects": "write"
    }
  ]
}
```

Tool Invocation via MCP (Claude → Gmail MCP Server)

After user confirmation, Claude executes the plan

- Claude does **not** call Gmail directly
- All execution is mediated by MCP
- Arguments are structured, not text-parsed

MCP tool call

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 2,
  "params": {
    "name": "gmail.send_email",
    "arguments": {
      "to": "luca@uni.it",
      "subject": "Meeting",
      "body": "Can we meet tomorrow at 10:00?"
    }
  }
}
```

Gmail API Execution (Inside MCP Server)

The MCP Server maps the tool call to the **real Gmail API**

(Relevant) Gmail API code

```
raw_email = create_mime_email(to, subject, body)

service.users().messages().send(
    userId="me",
    body={"raw": raw_email}
).execute()
```

MCP Server response

```
{
  "status": "SENT",
  "messageId": "1873abcf0",
  "threadId": "1873abbd1"
}
```

- Gmail API → MCP Server
- MCP Server → Claude Desktop
- Claude → User: **“Email sent ”**

Tool selection

Tool selection depends on:

- the **task**
- the **environment**
- the **AI model** powering the agent

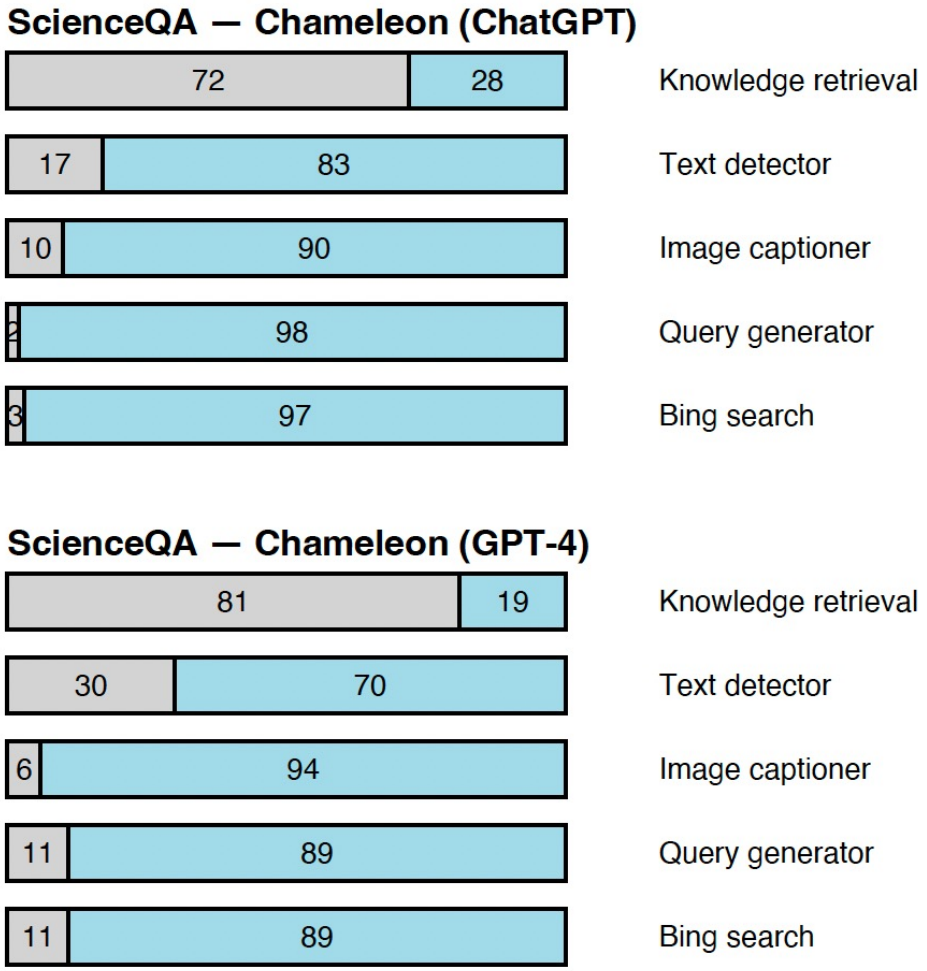
- There is **no foolproof recipe** for selecting the best tool set
- Tool selection is a **design decision**, not a solved problem
  - Key trade-off:
    - More tools ⇒ more capabilities
    - More tools ⇒ harder selection and usage
  - As the number of tools increases:
    - Tool descriptions become longer
    - Tool usage becomes harder to master (for agents and humans)
  - Tool descriptions may:
    - Exceed the model's **context window**
    - Confuse tool selection
  - Tool overload can **reduce performance**, despite higher theoretical capability

How to Evaluate and Refine Tool Selection

- Tool selection requires **experimentation and analysis**:
  - Compare agent performance with **different tool sets**
  - Perform **ablation studies**
    - Remove a tool
    - Measure performance drop
  - If removing a tool does not degrade performance, the tool is unnecessary and should be removed
- Identify tools the agent:
  - frequently misuses
  - struggles to learn even after prompting or fine-tuning

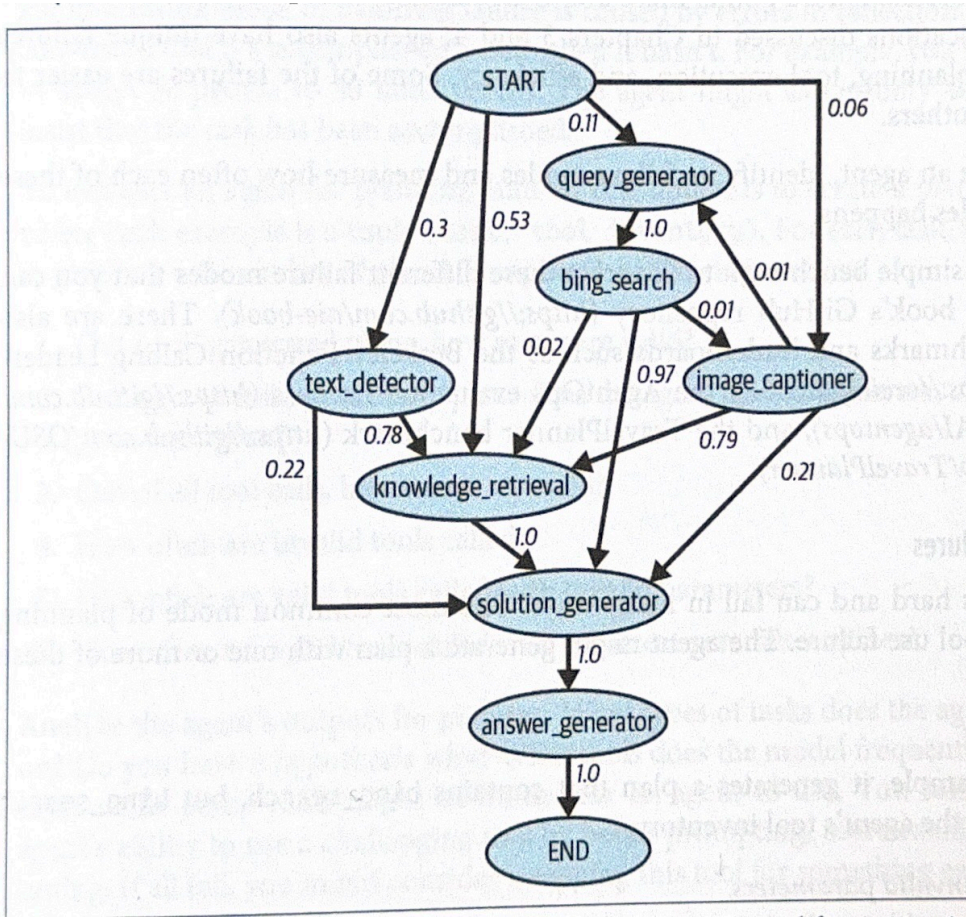
Tool Usage Patterns Depend on Task and Model

- Different tasks require **different tools**
  - ScienceQA → heavy use of **knowledge retrieval**
  - TabMWP → heavier use of **table reasoning tools**
- Different models show **different tool preferences**
  - GPT-4 tends to use a **wider variety of tools**
  - ChatGPT favors image captioning
  - GPT-4 favors knowledge retrieval
- Tool usage statistics reveal **implicit planning strategies**



Tool Transitions and Tool Composition

- Humans create **new tools by composing simpler ones**
  - Can agents do the same?
- **Chameleon (Lu et al., 2023)** studies *tool transitions*:
  - After tool X, how likely is tool Y?
- Observations:
  - Frequently co-used tools can be **merged into higher-level tools**
  - Tool usage forms **transition graphs**, not flat lists
- With awareness of transitions, agents can:
  - Build progressively more complex tools



Learning New Tools Over Time

- **Voyager (Wang et al., 2023)** introduces a *skill manager*
  - Each skill is executable code and corresponds to a reusable tool
- Workflow:
  - Agent solves a task
  - A useful new skill is discovered
  - Skill is added to a **skill library**
- Over time:
  - Tool inventory evolves
  - Agents accumulate **procedural knowledge**
- Tool selection and planning are **dynamic, not static**